SECURITY SELF-ASSESSMENT

Purse Autonomous Finance

Purse.ai

Architecture • Permission Tiers • Risk Register

Report scope

A structured presentation of the supplied security self-assessment, covering the application architecture, permission-tier execution model, reversibility and browser permissions, identified risks, and remediation recommendations.

Prepared from the supplied source document

This document organizes the provided assessment; it does not represent an independent penetration test or verification of the described controls.

Contents

1 Executive Summary and Architecture Scope **2**
1.1 Runtime and data-flow model 2

2 Permission-Tier Implementation **2**
2.1 Action lifecycle 3
2.2 Action safety tiers 3
2.3 Reversibility and rollback 4
2.4 Browser API and hardware permissions 4

3 Known-Risk Assessment **4**
3.1 Severity distribution 5

4 Summary Recommendation and Compliance Status **6**
4.1 Recommended remediation sequence 6

1 Executive Summary and Architecture Scope

System overview. Purse Autonomous Finance is described as a decentralized, local-first personal finance copilot. It combines an Express.js server for AI proxying and static serving with a client-side React 19 / TypeScript single-page application.

1.1 Runtime and data-flow model

- **Runtime:** Node.js Express server (`server.ts`) on port 3000, alongside a client-side Vite SPA.
- **Core data:** Accounts, transactions, bills, budgets, behavioral rules, and financial-profile memory are persisted in the user's browser.
- **AI processing:** Either local on-device inference using `SmolLM2-135M-Instruct-ONNX` through an in-browser ONNX runtime, or server-proxied cloud models such as Gemini, Claude, OpenAI, DeepSeek, and Groq.

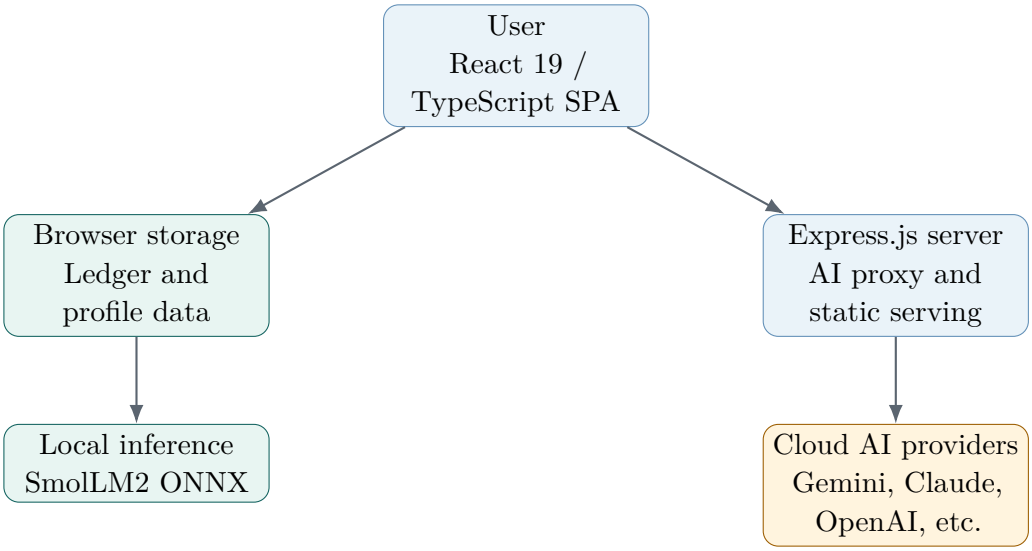


Figure 1: High-level architecture based on the supplied report. Arrows indicate described relationships, not a complete network trace.

2 Permission-Tier Implementation

The report describes a multi-layered permission and execution model intended to govern how agent recommendations can affect financial state.

2.1 Action lifecycle

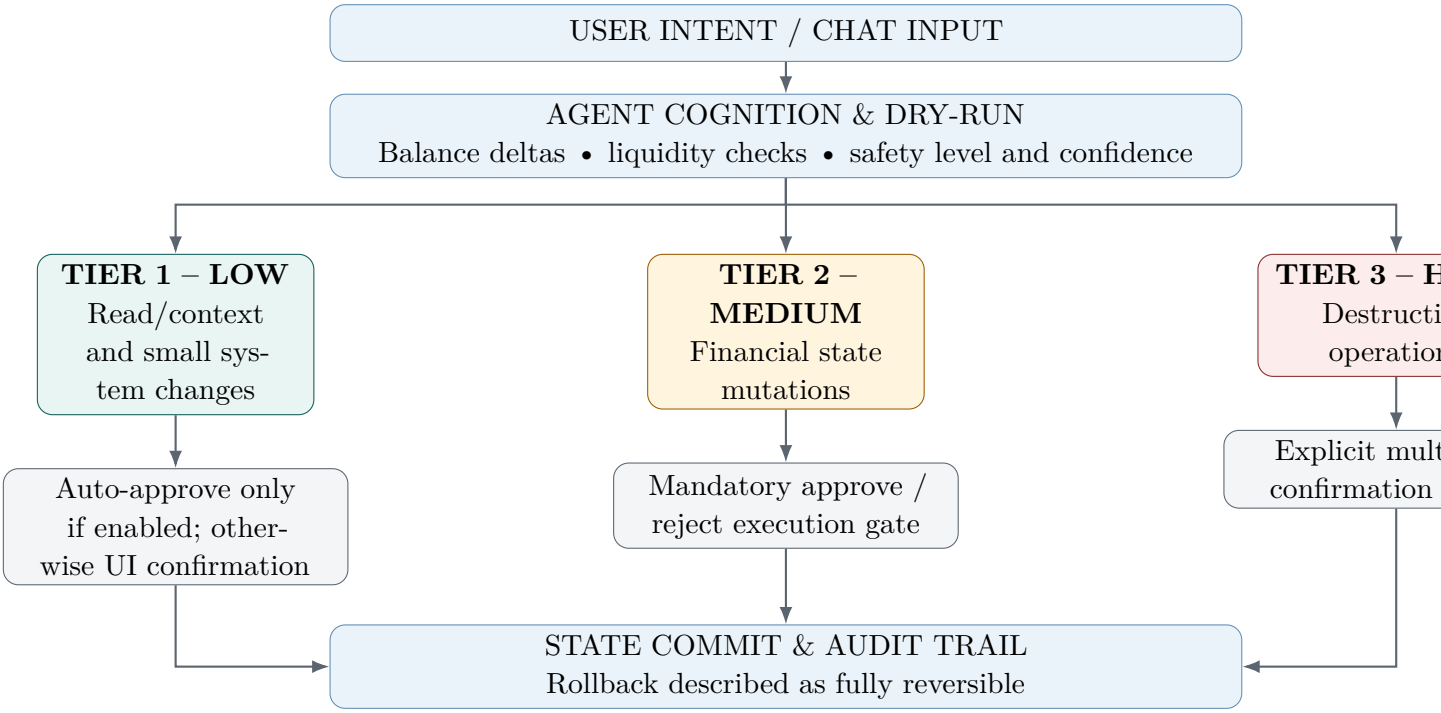


Figure 2: Permission and execution flow. Tier 0 read-only operations are described separately in Table 1.

2.2 Action safety tiers

Table 1: Action safety tiers described in the source report.

Tier	Classification	Example operations	Enforcement mechanism
0	Read-only / context retrieval	Balance queries, runway calculation, RAG vector searches, memory-graph inspection.	Synchronous execution with no state mutation; no execution gate required.
1	Low-risk system actions	Theme changes (`UPDATE\_SETTINGS`), voice mute/unmute, safe-to-spend calculations, minor metadata notes.	Sent to pending queue. Auto-commit only if `config.autoApproveSafeActions` is explicitly enabled by the user; otherwise UI confirmation.
2	Medium-risk financial mutations	`ADD\_TRANSACTION`, `EDIT\_TRANSACTION`, `DELETE\_TRANSACTION`, `ADD\_ACCOUNT`, `EDIT\_ACCOUNT`, `ADD\_BILL`, `PAY\_BILL`, `UPDATE\_BUDGET`.	Mandatory execution gate. Agent proposes an `AgentAction` with dry-run balance deltas and checking-floor warnings; user must approve or reject.

Tier	Classification	Example operations	Enforcement mechanism
3	High-risk destructive operations	`DELETE\_ACCOUNT` (with cascade/remapping), `DELETE\_MEMORY\_RULE`, `wipeEverything()`, `resetAllData()`.	Multi-step UI confirmation or modal verification. On account deletion, transaction history is described as remapped to the primary account to avoid orphan states.

2.3 Reversibility and rollback

- Each mutating action stores its `previousState` payload before application.
- `rollbackAction(actionId)` is described as reversing balance additions/subtractions, restoring soft-deleted entities, and returning the ledger to its exact pre-execution state.

2.4 Browser API and hardware permissions

Capability	Behavior described	Boundary / safeguard
Microphone access	`webkitSpeechRecognition` / Web Speech STT requests microphone permission on user activation.	Automated 90-second idle cutoff to prevent passive listening.
Persistent storage	`navigator.storage.persist` requests persistent client-storage quota.	Intended to reduce eviction risk under low-disk conditions.
File system access	CSV/JSON import and export use browser file-picker dialogs.	No direct disk access is described.

3 Known-Risk Assessment

The following risk register preserves the risk areas, severity labels, descriptions, existing mitigations, and recommendations supplied in the source.

Table 3: Risk register and recommended hardening actions.

ID	Risk area	Severity	Threat description	Current mitigation	Recommendation
R-01	Unauthenticated AI proxy endpoints	HIGH	`/api/chat` and `/api/organize-transactions` lack API session authentication or IP rate limiting. If `GEMINI\_API\_KEY` is configured, unauthenticated network clients could abuse server quota.	Express CORS origin checks; missing-key notice if no key is configured.	Add session tokens or JWT authentication to all `/api/*` routes and implement `express-rate-limit`.

ID	Risk area	Severity	Threat description	Current mitigation	Recommendation
R-02	Unencrypted client-side storage	MEDIUM	Ledger data (balances, masks, notes, transaction history) and vector embeddings are stored in plaintext `localStorage`; XSS or a rogue browser extension could inspect them.	No server storage of user ledgers; data scoped to the origin domain.	Move critical ledger storage to IndexedDB with Web Crypto API AES-GCM encryption and a key derived from a user passphrase.
R-03	Client-provided API key transmission	MEDIUM	User-provided OpenAI, Claude, DeepSeek, or Groq keys are sent in HTTP request bodies to `/api/chat` for proxying.	HTTPS in production; keys reportedly not logged to disk.	Keep BYOK keys exclusively client-side or call provider APIs directly from the browser where CORS permits.
R-04	Prompt injection via transaction ingestion	MEDIUM	Adversarial text in uploaded CSVs or pasted bank exports could influence AI parser output.	Parser expects strict JSON schemas for accounts, transactions, bills, and budgets; Execution Gate blocks automatic execution.	Apply strict schema validation (e.g., Zod) to parsed JSON before it enters ledger state.
R-05	Path traversal in custom static interceptor	LOW	Custom `server.ts` logic uses `path.join(publicDir, cleanPath)` to serve static files and icons before Vite fallback.	Leading slashes stripped from `cleanPath`; `fs.statSync().isFile()` check.	Enforce path containment after resolution; reject paths that resolve outside `publicDir`.
R-06	Third-party cloud PII exposure	LOW	Financial transaction notes and merchant names are sent to external AI APIs when cloud models are used.	Local `SmolLM2` ONNX option is described as offline, with no network calls.	Add a client-side PII scrubber to redact names and account digits before remote prompts are sent.

3.1 Severity distribution



Figure 3: Severity counts from the six risks listed in the supplied report: one High, three Medium, and two Low.

4 Summary Recommendation and Compliance Status

Current posture (as characterized by the source). The report describes the Execution Gate and reversible-action model as strong client-side boundary defenses against unintended autonomous writes.

Immediate remediation priority. Secure the Express routes `/api/chat` and `/api/organize-transactions` with rate limiting and origin/session authentication to reduce the risk of API quota abuse.

4.1 Recommended remediation sequence

1. **First: protect API routes.** Add session or JWT authentication and rate limiting to the AI proxy and transaction-organization endpoints.
2. **Next: protect sensitive browser data.** Evaluate encrypted client-side storage and strict validation of imported transaction data.
3. **Then: reduce key and PII exposure.** Revisit BYOK key handling and redact sensitive transaction details before sending prompts to cloud AI providers.
4. **Harden supporting boundaries.** Add explicit resolved-path containment checks to the custom static-file interceptor.